

Prova em grupo — Construa o backend do seu sistema

Disciplina: Programação Orientada a Objetos **Formato:** entrega em grupo, tema sorteado
Grupos: 3 a 4 pessoas **Prazo:** 7 dias corridos a partir do sorteio **Repositório modelo:** [kioferta-api](#) (use como referência de estrutura)

1. O que é esta prova

Cada grupo recebe, por **sorteio**, o tema de um sistema. O trabalho é construir o **backend** desse sistema do zero, seguindo a mesma arquitetura em quatro camadas do repositório modelo.

O escopo é propositalmente **menor que o KiOferta**: três a quatro classes, uma hierarquia de herança, e de cinco a sete rotas. Dá para fazer em uma semana sem virar noite.

O que está sendo avaliado não é o tamanho do sistema. É se você consegue:

- decidir **quais classes existem** no domínio que recebeu
- proteger o estado delas e colocar a **regra no lugar certo**
- usar **herança** onde ela se justifica, e **polimorfismo** para evitar **if**
- separar **data, models, controllers e routes** sem vazamento
- traduzir erro de negócio em **resposta HTTP**

Por que um tema diferente por grupo

Porque o domínio muda, mas a estrutura não. Quando você vir que a sua clínica veterinária e a locadora do grupo vizinho têm exatamente o mesmo esqueleto, o padrão deixa de ser uma receita decorada e vira uma ferramenta.

2. Os 12 temas

Todos seguem a mesma forma: **uma entidade principal, uma hierarquia, um registro que liga as duas**.

Tema 1 — Biblioteca comunitária

Controle de acervo e empréstimos de um ponto de leitura de bairro.

Camada	O que criar
Entidades	Livro, Empréstimo
Hierarquia	Pessoa → Leitor → Bibliotecario
Liga as duas	Empréstimo guarda um Livro e uma Pessoa

Regras obrigatórias

- Leitor pode ter no máximo 3 empréstimos; bibliotecário, 10 (constante de classe)
- Não é possível emprestar um livro já emprestado
- O ano de publicação não pode ser maior que o ano atual
- Só quem tem a permissão `cadastrar_livro` pode incluir um título novo

Rotas mínimas

GET /api/livros	lista o acervo
GET /api/livros/{id}	um livro, ou 404
GET /api/livros/disponiveis	só os que não estão emprestados
POST /api/emprestimos	registra, 201 ou 409
GET /api/pessoas/{id}/emprestimos	o que a pessoa está com
POST /api/login	perfil e permissões

Tema 2 — Clínica veterinária

Agendamento de consultas de uma clínica de bairro.

Camada	O que criar
Entidades	Tutor, Consulta
Hierarquia	Animal → Cao, Gato, Ave
Liga as duas	Consulta guarda um Animal e um Veterinario

Regras obrigatórias

- Cada espécie tem um valor de consulta diferente (constante de classe)
- Animal tem um método descricao() que cada espécie completa com super()
- A idade do animal não pode ser negativa nem maior que 30
- Não pode haver duas consultas no mesmo horário para o mesmo veterinário

Rotas mínimas

GET /api/animais lista

GET /api/animais/{id} um animal, ou 404

GET /api/animais/especie/{especie} filtra por espécie

POST /api/consultas agenda, 201 ou 409

GET /api/animais/{id}/consultas histórico do animal

GET /api/consultas/valor-total soma do dia

Tema 3 — Academia

Controle de alunos, planos e entradas na catraca.

Camada	O que criar
Entidades	Aluno, Checkin
Hierarquia	Plano → Mensal, Trimestral, Anual
Liga as duas	Checkin guarda um Aluno e uma data

Regras obrigatórias

- Cada plano tem um valor e um desconto próprios; `valor_final()` usa `super()`
- O plano Mensal permite 12 entradas por mês; o Anual, ilimitado
- O aluno não pode fazer dois checkins no mesmo dia
- Idade mínima de 14 anos

Rotas mínimas

GET /api/alunos	lista
GET /api/alunos/{id}	um aluno, com o plano, ou 404
GET /api/planos	planos e valores finais
POST /api/checkins	registra, 201 ou 409
GET /api/alunos/{id}/checkins	histórico
GET /api/relatorio/faturamento	soma dos planos ativos

Tema 4 — Estacionamento

Controle de entrada, saída e cobrança.

Camada	O que criar
Entidades	<code>Vaga</code> , <code>Ticket</code>
Hierarquia	<code>Veiculo</code> → <code>Carro</code> , <code>Moto</code> , <code>Caminhonete</code>
Liga as duas	<code>Ticket</code> guarda um <code>Veiculo</code> e uma <code>Vaga</code>

Regras obrigatórias

- Cada tipo de veículo tem tarifa por hora diferente (constante de classe)
- Moto não pode ocupar vaga de carro, e vice-versa
- Placa precisa ter 7 caracteres
- Não é possível dar entrada em vaga já ocupada

Rotas mínimas

GET /api/vagas	todas, com ocupação
GET /api/vagas/livres	só as livres
POST /api/tickets	entrada, 201 ou 409
POST /api/tickets/{id}/saida	calcula o valor, 200 ou 404
GET /api/tickets/abertos	quem está dentro
GET /api/relatorio/faturamento	total do dia

Tema 5 — Cinema

Venda de ingressos de um cinema de rua.

Camada	O que criar
Entidades	Filme, Sessao
Hierarquia	Ingresso → Inteira, Meia, Promocional
Liga as duas	Ingresso guarda uma Sessao

Regras obrigatórias

- Cada tipo de ingresso calcula o próprio `valor()` a partir do preço da sessão
- A sessão tem capacidade máxima; não se vende ingresso além dela
- `Meia` exige um documento informado, senão recusa
- A classificação indicativa do filme não pode ser negativa

Rotas mínimas

GET /api/filmes	lista
GET /api/sesoes	sessões com lugares restantes
GET /api/sesoes/{id}	uma sessão, ou 404

POST /api/ingressos vende, 201 ou 409

GET /api/sessoes/{id}/ingressos vendidos daquela sessão

GET /api/relatorio/bilheteria total por filme

Tema 6 — Pet shop: banho e tosa

Agenda de serviços de um pet shop.

Camada	O que criar
Entidades	Pet, Agendamento
Hierarquia	Servico → Banho, Tosa, BanhoETosa
Liga as duas	Agendamento guarda um Pet e um Servico

Regras obrigatórias

- BanhoETosa herda de Tosa e soma a duração do banho com `super()`
- O preço varia pelo porte do pet, calculado dentro do serviço
- Não pode haver dois agendamentos no mesmo horário
- O porte precisa ser pequeno, médio ou grande

Rotas mínimas

GET /api/pets lista

GET /api/servicos serviços, preços e durações

POST /api/agendamentos agenda, 201 ou 409

GET /api/agendamentos/dia/{data} agenda do dia

GET /api/pets/{id}/agendamentos histórico do pet

DELETE /api/agendamentos/{id} cancela, 200 ou 404

Tema 7 — Oficina mecânica

Ordens de serviço de uma oficina.

Camada	O que criar
Entidades	<code>Veiculo</code> , <code>OrdemServico</code>
Hierarquia	<code>Item</code> → <code>MaoDeObra</code> , <code>Peca</code>
Liga as duas	<code>OrdemServico</code> guarda um <code>Veiculo</code> e uma lista de <code>Item</code>

Regras obrigatórias

- `MaoDeObra` cobra por hora; `Peca` cobra por unidade. Cada um tem seu `total()`
- A ordem calcula o valor somando os itens, sem saber o tipo de cada um
- Ordem fechada não aceita item novo
- Quantidade e horas precisam ser maiores que zero

Rotas mínimas

GET <code>/api/veiculos</code>	lista
GET <code>/api/ordens</code>	ordens abertas e fechadas
GET <code>/api/ordens/{id}</code>	uma ordem com os itens e o total
POST <code>/api/ordens/{id}/itens</code>	acrescenta item, 201 ou 409
POST <code>/api/ordens/{id}/fechar</code>	fecha, 200 ou 409
GET <code>/api/relatorio/faturamento</code>	total das ordens fechadas

Tema 8 — Restaurante com delivery

Pedidos de um restaurante pequeno.

Camada	O que criar
Entidades	Prato, Pedido
Hierarquia	Entrega → Retirada, Delivery, DeliveryExpresso
Liga as duas	Pedido guarda uma lista de Prato e uma Entrega

Regras obrigatórias

- Cada forma de entrega calcula a própria taxa; DeliveryExpresso usa `super()` e soma
- Pedido abaixo do mínimo não aceita delivery
- Pedido sem nenhum prato não pode ser criado
- O total soma os pratos mais a taxa, sem `if` de tipo de entrega

Rotas mínimas

GET /api/pratos cardápio

GET /api/pratos/categoria/{cat} filtra

POST /api/pedidos cria, 201 ou 422

GET /api/pedidos/{id} um pedido com o total, ou 404

GET /api/pedidos todos

GET /api/relatorio/ticket-medio média dos pedidos

Tema 9 — Locadora de equipamentos

Aluguel de ferramentas e equipamentos.

Camada	O que criar
Entidades	Equipamento, Locacao

Camada	O que criar
Hierarquia	Cliente → PessoaFisica, PessoaJuridica
Liga as duas	Locacao guarda um Equipamento e um Cliente

Regras obrigatórias

- PessoaJuridica tem desconto e limite de locações diferente (constante de classe)
- Equipamento já locado não pode ser locado de novo
- CPF precisa ter 11 dígitos; CNPJ, 14. A validação fica em cada filha
- Diária não pode ser negativa

Rotas mínimas

GET /api/equipamentos lista

GET /api/equipamentos/disponiveis só os livres

POST /api/locacoes aluga, 201 ou 409

POST /api/locacoes/{id}/devolver devolve e calcula, 200 ou 404

GET /api/clientes/{id}/locacoes histórico

GET /api/relatorio/faturamento total

Tema 10 — Doação de alimentos

Ponto de coleta de uma campanha solidária.

Camada	O que criar
Entidades	PontoColeta, Doacao
Hierarquia	Doador → DoadorPessoa, DoadorEmpresa, DoadorParceiro

Camada	O que criar
Liga as duas	<code>Doacao</code> guarda um <code>Doador</code> e um <code>PontoColeta</code>

Regras obrigatórias

- Cada tipo de doador tem uma pontuação diferente por quilo doado
- `DoadorParceiro` herda de `DoadorEmpresa` e soma um bônus com `super()`
- Doação com peso zero ou negativo é recusada
- O ponto de coleta tem capacidade máxima em quilos

Rotas mínimas

GET `/api/pontos` pontos e quanto já receberam

GET `/api/pontos/{id}` um ponto, ou 404

POST `/api/doacoes` registra, 201 ou 409

GET `/api/doadores/ranking` ordenado pela pontuação

GET `/api/doadores/{id}/doacoes` histórico

GET `/api/relatorio/total` quilos arrecadados

Tema 11 — Escola de idiomas

Matrículas em turmas de um curso livre.

Camada	O que criar
Entidades	<code>Turma</code> , <code>Matricula</code>
Hierarquia	<code>Curso</code> → <code>Presencial</code> , <code>Online</code> , <code>Hibrido</code>
Liga as duas	<code>Matricula</code> guarda um <code>Aluno</code> e uma <code>Turma</code>

Regras obrigatórias

- Cada modalidade calcula a mensalidade de forma diferente; `Hibrido` usa `super()`

- A turma tem vagas limitadas; matrícula além disso é recusada
- O mesmo aluno não se matricula duas vezes na mesma turma
- A carga horária precisa ser maior que zero

Rotas mínimas

GET /api/cursos	cursos e mensalidades
GET /api/turmas	turmas com vagas restantes
GET /api/turmas/{id}	uma turma, ou 404
POST /api/matriculas	matricula, 201 ou 409
GET /api/alunos/{id}/matriculas	o que o aluno cursa
GET /api/relatorio/ocupacao	percentual de vagas por turma

Tema 12 — Carona solidária no campus

Oferta e reserva de caronas entre alunos.

Camada	O que criar
Entidades	Viagem, Reserva
Hierarquia	Participante → Passageiro, Motorista, MotoristaVerificado
Liga as duas	Reserva guarda uma Viagem e um Passageiro

Regras obrigatórias

- Só quem tem a permissão `criar_viagem` pode oferecer carona
- `MotoristaVerificado` herda de `Motorista` e acrescenta permissões com `super()`
- A viagem tem número de lugares; reserva além disso é recusada
- O mesmo passageiro não reserva a mesma viagem duas vezes

Rotas mínimas

GET /api/viagens	viagens com lugares restantes
GET /api/viagens/{id}	uma viagem, ou 404
POST /api/viagens	cria, 201 ou 403
POST /api/reservas	reserva, 201 ou 409
GET /api/participantes/{id}/reservas	histórico
POST /api/login	perfil e permissões

3. O que todo grupo precisa entregar

Independentemente do tema sorteado.

Estrutura de pastas

nome-do-sistema/

├── main.py

├── requirements.txt

├── verificar.py

├── README.md

└── app/

 ├── __init__.py

 ├── data/ mocks, um arquivo por entidade

 ├── models/ classes e regras

 ├── controllers/ casos de uso

 └── routes/ endereços e códigos HTTP

Não invente outra organização. Copie a do repositório modelo.

Camada data

- ☐ Um arquivo `*_mock.py` por entidade, com lista de dicionários
- ☐ Pelo menos **5 registros** por entidade principal
- ☐ Nenhuma classe, nenhum import, nenhuma regra nesses arquivos

Camada models

- ☐ De **3 a 5 classes**, contando a hierarquia
- ☐ Todos os atributos protegidos, com um sublinhado
- ☐ Um `mostrar_` para cada leitura necessária
- ☐ `alterar_` só onde o valor pode mudar; **nenhum `alterar_id`**
- ☐ O construtor chama os `alterar_`
- ☐ **No mínimo 4 regras de negócio** com `raise ValueError`
- ☐ `__repr__` em todas as classes
- ☐ Uma função `carregar_*`() por entidade, no fim do arquivo
- ☐ **Nenhum import do FastAPI**

Herança e polimorfismo

- ☐ Uma hierarquia com **no mínimo 3 classes e 2 níveis**
- ☐ Pelo menos **um método sobrescrito** que usa `super()` para estender
- ☐ Pelo menos **uma constante de classe** com valor diferente nas filhas
- ☐ Um dicionário no estilo `PERFIS`, mapeando texto do mock para classe
- ☐ **Nenhum `if` comparando tipo ou nome de classe** em todo o projeto

Camada controllers

- ☐ Um controller por entidade principal
- ☐ Um método por caso de uso
- ☐ `_para_dicionario` convertendo o objeto
- ☐ Pelo menos **um filtro** com compreensão de lista
- ☐ Devolve `None` ou lista vazia quando não encontra; nunca código HTTP

Camada routes

- ☐ De **5 a 7 rotas**, conforme o tema
- ☐ `404` quando não encontra
- ☐ `409` quando conflita com o estado atual
- ☐ `422` quando a regra de negócio é violada

- ☐ 201 na criação
- ☐ try/except traduzindo o ValueError da model

Verificação

- ☐ Um verificar.py com no mínimo 12 checagens, no formato do modelo
- ☐ Ele precisa passar inteiro antes da entrega

README

- ☐ Como rodar
 - ☐ O diagrama de classes, mesmo que em texto ou foto de desenho à mão
 - ☐ A tabela de rotas
 - ☐ Quem fez o quê
-

4. Diagrama de classes

Entregue um diagrama UML com:

- as 3 a 5 classes, com atributos e métodos
- os sinais - para protegido e + para público
- o triângulo vazio da herança
- a linha da associação, com as multiplicidades

Pode ser feito no Draw.io, no Mermaid dentro do README, ou à mão e fotografado. O que vale é estar correto, não bonito.

5. Prazo e entrega

Etapa	Quando
Sorteio dos temas	em aula
Dúvidas por issue no repositório da turma	até o 5º dia
Entrega	7º dia, às 23h59
Apresentação e arguição	aula seguinte

Como entregar

1. Um repositório público por grupo, no GitHub
 2. Commits de todos os integrantes ao longo da semana, e não um commit único no fim
 3. O link na planilha da turma
 4. Na raiz: o `README.md` e a saída do `verificar.py` colada nele
-

6. Apresentação

8 minutos por grupo, mais 4 de perguntas.

Tempo	O quê
1 min	O domínio: que sistema é esse e para quem serve
2 min	O diagrama de classes e por que vocês modelaram assim
3 min	A API rodando ao vivo no <code>/docs</code> , incluindo um caso de erro
2 min	Onde está a herança e onde está o polimorfismo, mostrando o código

Todos falam. A arguição é individual: qualquer integrante pode ser chamado a explicar qualquer linha.

7. Rubrica — 100 pontos

Critério	Pontos	O que é avaliado
Modelagem do domínio	15	as classes fazem sentido para o tema; a hierarquia se justifica
Encapsulamento	20	protegidos, construtor validando, sem

Critério	Pontos	O que é avaliado
		<code>alterar_id</code> , regra na model
Herança e polimorfismo	20	<code>super()</code> estendendo, constante de classe, zero <code>if</code> de tipo
Camadas	15	sem vazamento; a model não conhece o FastAPI
Coleções	10	pelo menos um filtro com compreensão de lista
Rotas e códigos HTTP	10	404, 409, 422 e 201 nos lugares certos
Diagrama e README	5	diagrama correto, instruções que funcionam
Apresentação e arguição	5	todos falam e sabem explicar o código

Penalidades

Falha	Perda
<code>import</code> do FastAPI dentro de <code>models/</code>	metade dos pontos de camadas
<code>if</code> comparando tipo ou nome de classe	metade dos pontos de herança
Validação escrita na rota em vez da model	metade dos pontos de encapsulamento
<code>return 'mensagem de erro'</code> em vez de <code>raise</code>	metade dos pontos de encapsulamento
Um único commit, ou commits de uma pessoa só	metade dos pontos de apresentação
A API não sobe	zero nos pontos de rotas

8. Perguntas da arguição

Prepare-se para responder sobre **o seu próprio código**:

1. Por que vocês escolheram essas classes, e não outras?
 2. Onde está a herança? Por que ela se justifica aqui?
 3. Mostre um método que usa `super()` e explique o que acontece se tirá-lo.
 4. Onde está o polimorfismo? Que `if` ele evitou?
 5. Por que essa constante está na classe filha e não num `if` no controller?
 6. Se eu criar uma quarta subclasse, quantos arquivos vocês mexeriam?
 7. Por que esta regra está na model e não no controller?
 8. A model importa alguma coisa do FastAPI? Por que isso importa?
 9. Se os mocks virassem banco de dados, quais arquivos mudariam?
 10. Qual é o tipo de relacionamento entre essas duas classes, e por quê?
-

9. Perguntas frequentes

Podemos usar biblioteca externa? Só FastAPI e Uvicorn. Nada de banco de dados, ORM ou autenticação com token.

Podemos usar inteligência artificial? Podem, e é bom que saibam usar. Mas a arguição é individual: código que o grupo não sabe explicar vale zero, mesmo funcionando.

Precisa ter tela? Não. É backend. A prova de que funciona é o `/docs`.

Podemos trocar de tema? Não. O sorteio é a prova.

E se o tema parecer grande demais? Entregue o mínimo pedido nas rotas. Funcionalidade extra não vale ponto adicional; qualidade de modelagem vale.

E se não terminarmos? Entreguem o que fizeram. Models e controllers prontos sem as rotas valem mais do que nada. Mas abram a issue antes do prazo: quase sempre dá para destravar.

10. Checklist final

- ☐ A API sobe com `uvicorn main:app --reload`
- ☐ As rotas aparecem no `/docs`

- ☐ Nenhum `import` de FastAPI dentro de `app/models/`
- ☐ Nenhum `if` comparando tipo ou nome de classe
- ☐ A hierarquia tem 3 classes e 2 níveis
- ☐ Existe um método com `super()` estendendo o da base
- ☐ Existe uma constante de classe com valor diferente nas filhas
- ☐ Existem no mínimo 4 regras com `raise`
- ☐ O construtor chama os métodos `alterar_`
- ☐ Não existe `alterar_id`
- ☐ Existe pelo menos um filtro com compreensão de lista
- ☐ As rotas devolvem 404, 409, 422 e 201
- ☐ O `verificar.py` passa em todas as checagens
- ☐ O diagrama de classes está no README
- ☐ Todos os integrantes têm commits
- ☐ Todos sabem explicar todo o código